The Journal of Open Source Software

PINNeAPPle: A Python platform for physics-informed machine learning with physics-based trust checks

Yan Barros [1¶] **and Enrico Schiassi** [2,3]

1 PINNeAPPle Labs, Brazil **2** University of Arizona, Tucson, AZ, United States **3** University of Bologna, Bologna, Italy ¶ Corresponding author

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 13 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Many questions in science and engineering come down to solving equations that describe how a physical quantity, such as temperature, velocity or stress, changes in space and time. Neural networks can now approximate such solutions quickly, either by being trained against the equations themselves or by learning from many earlier simulations. Their weak point is trust: a network always returns an answer, even when the case in front of it lies outside what it has learned.

PINNeAPPle is an open-source Python library, built on PyTorch (Ansel et al., 2024), that brings these methods together under one interface and checks their predictions against the governing physics. A user describes a physical problem once and solves it with an exact solution, a classical numerical method or a neural model, then compares the results on the same footing. The library measures how well each prediction satisfies the governing equations, conservation laws and boundary conditions, and uses this to decide which model to use on each case and to flag when a model should not be trusted.

Statement of need

Physics-informed neural networks (PINNs) (Raissi et al., 2019), neural operators such as DeepONet (Lu, Jin, et al., 2021) and the Fourier neural operator (FNO) (Li et al., 2021), and graph networks on meshes (Pfaff et al., 2021) are now standard tools of physics-informed machine learning (Karniadakis et al., 2021). Researchers who use them face two practical problems. First, each family lives in its own library with its own problem description, so a fair comparison between a PINN, an operator, a graph network and a classical solver on the same problem takes substantial glue code. Second, these models are usually judged by their error against a reference solution, which is not available when the model is used on new cases.

PINNeAPPle is written for researchers and engineers in computational science who train or compare physics-based surrogate models and need to know when those models stop being valid. It addresses the two problems with (i) a single problem description that every method consumes, together with a comparison interface and a record of each run, and (ii) a set of physics-based checks that need no reference solution: the residual of the governing equations, physical bounds, conservation and the validity conditions that each model declares.

State of the field

Several mature libraries cover parts of this space. DeepXDE (Lu, Meng, et al., 2021) is the reference implementation for PINNs and DeepONet, with a wide range of geometries, boundary conditions and training strategies. NVIDIA PhysicsNeMo (PhysicsNeMo Contributors,

DRAFT

2023) targets large-scale, GPU-accelerated training of PINNs, operators and graph networks. NeuralPDE.jl (Zubov et al., 2021) generates PINNs from symbolic PDE descriptions in Julia. PINA (Coscia et al., 2023) offers a PyTorch Lightning interface for PINNs and neural operators. SciANN (Haghighat & Juanes, 2021) wraps Keras/TensorFlow for physics-informed learning, IDRLnet (Peng et al., 2021) provides a PINN library built on PyTorch, and neuraloperator (Kossaifi et al., 2024) is the reference library for FNO and related operators.

PINNeAPPle does not try to replace these libraries in their core tasks: for a large PINN, DeepXDE or PhysicsNeMo will usually be the more mature choice, and neuraloperator for operator learning at scale. Instead of rebuilding them, PINNeAPPle provides bridges: a DeepXDE adapter that turns a PINNeAPPle problem into a DeepXDE model and trains it with DeepXDE itself, and a PhysicsNeMo bridge for models trained there. What the libraries above do not provide, and the reason PINNeAPPle was built as a separate package rather than as a contribution to one of them, is a layer that sits across model families: online selection and combination of PINNs, operators, graph networks, solvers and closed forms from their physics residuals, a physics audit of each model's assumptions, and a common record of runs and checks. These functions depend on treating very different models as interchangeable experts, which is outside the scope of any single-family library. PINNeAPPle also implements the extreme theory of functional connections (X-TFC) (Schiassi et al., 2021), which few general-purpose libraries include.

Software design

The library is organised as a set of Python packages around three shared abstractions. A `PhysicalProblem` holds the equations, domain, parameters and conditions of a problem. Solving methods are registered by name with `register_method` and are called through `solve` and `compare`, so a new method is compared against existing ones without changing them. An `Experiment` records a run with a fingerprint of the problem, so results stay attached to the exact problem that produced them.

Two design choices follow from the goal of judging models by their physics. First, the ensemble and the physics audit are written in plain NumPy and see every model only through a prediction function, so PINNs, operators, graph networks, numerical solvers and closed-form solutions are handled the same way. Second, model weights in the ensemble are updated only from cases already seen (a prequential Fixed-Share and AdaHedge scheme), so the choice of model cannot overfit the evaluation data; point-wise intervals are calibrated online by adaptive conformal inference. The trade-off is that the ensemble does not exploit model internals, such as gradients, that a single-family library could use.

Heavy dependencies (OpenFOAM, CalculiX, FEniCS, DeepXDE, PhysicsNeMo and 3-D viewers) are optional extras, so the core installs with NumPy, SciPy and PyTorch, and tests that need an optional backend are skipped when it is absent.

Key features

- **Neural models:** PINN variants (including XPINN, VPINN and Bayesian PINNs), DeepONet, FNO, physics-informed neural operators, MeshGraphNet, SIREN, physics-informed extreme learning machines and sparse identification of nonlinear dynamics (SINDy) (Brunton et al., 2016).
- **X-TFC solvers:** constrained expressions from the theory of functional connections (Leake et al., 2020) combined with extreme learning machines and least-squares or Gauss-Newton solves, for systems of ODE initial-value problems, for 1-D heat and Burgers equations and on subdomains (Schiassi et al., 2021).
- **Symbolic PDEs:** SymPy expressions compiled into PyTorch autograd residuals with hard or soft boundary conditions.

DRAFT

- **Physics ensembles:** `PhysicsEnsemble` selects or mixes experts online from reference errors, physics residuals and cost, and can detect a regime change from the residual before any reference is available.
- **Physics audit:** monitors residuals, conservation, bounds and extrapolation, checks each model's declared assumptions against dimensionless numbers, and attributes failures to a probable cause.
- **Classical solvers and bridges:** finite differences, finite elements, finite volumes, spectral and lattice Boltzmann methods, with interfaces to OpenFOAM, CalculiX, FEniCS, DeepXDE and PhysicsNeMo.
- **Experiment records:** the PINNeAPPle Lab stores each run with its code, data, checks against references and figures, and builds an HTML catalogue.

Research impact statement

PINNeAPPle is distributed on PyPI (`pip install pinneapple`) under the Apache-2.0 licence and has had tagged releases since version 0.5.0. Its automated test suite runs on every pull request. The repository publishes a validation status table, generated from the tests, which marks a method as validated only when at least one test compares it with an independent reference (a closed-form or manufactured solution, or a published value), and lists the methods that are tested without such a reference or not tested at all. For example, the X-TFC solvers are validated against exact solutions, and the README example of a physics ensemble switches models on the first case of a new regime of an advection-diffusion problem, using the PDE residual and no reference.

The Lab includes reproducible experiments whose data, code and checks are stored in the repository, including recovering the pendulum law from video, Kepler's third law from planetary data and the Lorenz parameters from noisy data, and checking finite-element and OpenFOAM results against analytical solutions, engineering correlations and CalculiX. The repository has received issues and pull requests from contributors outside the author team.

Beyond the Lab, PINNeAPPle is the basis of the PINNeAPPle-Benchmark repository (https://github.com/PINNeAPPle-Labs/PINNeAPPle-Benchmark), where each research experiment ships its code, results and a technical report generated from them: a physics-informed digital twin with parameter identification and anomaly detection; a crash surrogate for a bumper beam trained on an OpenRadioss design of experiments, reproducing and then improving a Transolver baseline; constraint-aware terramechanics surrogates; a heated-channel digital twin comparing FNO, DeepONet and POD surrogates with ensemble Kalman filtering; a reproduction and improvement of a published pedestrian dead-reckoning thesis; and a supersonic shock train in a duct. As of October 2026 the repository has external contributors and 17 forks.

AI usage disclosure

Generative AI tools were used in the development of PINNeAPPle. Claude Code (Anthropic), an agentic coding assistant running Claude models (Opus family), and ChatGPT (OpenAI) were used to write and refactor source code, tests, examples and documentation, to run and analyse simulations and benchmark experiments, and to draft parts of this paper. The human authors defined the scope, the architecture and the scientific choices of the library, reviewed and edited the assisted output, and validated the numerical methods against analytical solutions, independent implementations and reference data, as described in the repository's tests and experiments. The authors take full responsibility for the content of the software and of this paper.

DRAFT

Acknowledgements

We thank everyone who has contributed code, issues and reviews to PINNeAPPle.

This work received no specific grant from any funding agency in the public, commercial or not-for-profit sectors.

References

Ansel, J., Yang, E., He, H., Gimelshein, N., Jain, A., Voznesensky, M., Bao, B., Bell, P., Berard, D., Burovski, E., Chauhan, G., Chourdia, A., Constable, W., Desmaison, A., DeVito, Z., Ellison, E., Feng, W., Gong, J., Gschwind, M., … Chintala, S. (2024). PyTorch 2: Faster machine learning through dynamic Python bytecode transformation and graph compilation. *29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2 (ASPLOS '24)*. https://doi.org/10.1145/3620665.3640366

Brunton, S. L., Proctor, J. L., & Kutz, J. N. (2016). Discovering governing equations from data by sparse identification of nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, *113*(15), 3932–3937. https://doi.org/10.1073/pnas.1517384113

Coscia, D., Ivagnes, A., Demo, N., & Rozza, G. (2023). Physics-informed neural networks for advanced modeling. *Journal of Open Source Software*, *8*(87), 5352. https://doi.org/10.21105/joss.05352

Haghighat, E., & Juanes, R. (2021). SciANN: A Keras/TensorFlow wrapper for scientific computations and physics-informed deep learning using artificial neural networks. *Computer Methods in Applied Mechanics and Engineering*, *373*, 113552. https://doi.org/10.1016/j.cma.2020.113552

Karniadakis, G. E., Kevrekidis, I. G., Lu, L., Perdikaris, P., Wang, S., & Yang, L. (2021). Physics-informed machine learning. *Nature Reviews Physics*, *3*, 422–440. https://doi.org/10.1038/s42254-021-00314-5

Kossaifi, J., Kovachki, N., Li, Z., Pitt, D., Liu-Schiaffini, M., George, R. J., Bonev, B., Azizzadenesheli, K., Berner, J., Duruisseaux, V., & Anandkumar, A. (2024). *A library for learning neural operators*. https://doi.org/10.48550/arXiv.2412.10354

Leake, C., Johnston, H., & Mortari, D. (2020). The multivariate theory of functional connections: Theory, proofs, and application in partial differential equations. *Mathematics*, *8*(8), 1303. https://doi.org/10.3390/math8081303

Li, Z., Kovachki, N., Azizzadenesheli, K., Liu, B., Bhattacharya, K., Stuart, A., & Anandkumar, A. (2021). Fourier neural operator for parametric partial differential equations. *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2010.08895

Lu, L., Jin, P., Pang, G., Zhang, Z., & Karniadakis, G. E. (2021). Learning nonlinear operators via DeepONet based on the universal approximation theorem of operators. *Nature Machine Intelligence*, *3*(3), 218–229. https://doi.org/10.1038/s42256-021-00302-5

Lu, L., Meng, X., Mao, Z., & Karniadakis, G. E. (2021). DeepXDE: A deep learning library for solving differential equations. *SIAM Review*, *63*(1), 208–228. https://doi.org/10.1137/19M1274067

Peng, W., Zhang, J., Zhou, W., Zhao, X., Yao, W., & Chen, X. (2021). *IDRLnet: A physics-informed neural network library*. https://doi.org/10.48550/arXiv.2107.04320

Pfaff, T., Fortunato, M., Sanchez-Gonzalez, A., & Battaglia, P. W. (2021). Learning mesh-based simulation with graph networks. *International Conference on Learning Representations*. https://doi.org/10.48550/arXiv.2010.03409

DRAFT

PhysicsNeMo Contributors. (2023). *NVIDIA PhysicsNeMo: An open-source framework for physics-based deep learning in science and engineering*. https://github.com/NVIDIA/physicsnemo

Raissi, M., Perdikaris, P., & Karniadakis, G. E. (2019). Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, *378*, 686–707. https://doi.org/10.1016/j.jcp.2018.10.045

Schiassi, E., Furfaro, R., Leake, C., De Florio, M., Johnston, H., & Mortari, D. (2021). Extreme theory of functional connections: A fast physics-informed neural network method for solving ordinary and partial differential equations. *Neurocomputing*, *457*, 334–356. https://doi.org/10.1016/j.neucom.2021.06.015

Zubov, K., McCarthy, Z., Ma, Y., Calisto, F., Pagliarino, V., Azeglio, S., Bottero, L., Luján, E., Sulzer, V., Bharambe, A., Vinchhi, N., Balakrishnan, K., Upadhyay, D., & Rackauckas, C. (2021). *NeuralPDE: Automating physics-informed neural networks (PINNs) with error approximations*. https://doi.org/10.48550/arXiv.2107.09443

DRAFT